

2st Sessional
Deep Learning (BCSML0555A/BCSML0552)
Session: 2026-27 (V Sem.)

Total Marks:30
Section A Quize:10X1=10
Section B Marks :4X5=20
Date:20/07/2026

General Instructions

1. Attempt all four questions. All questions carry equal marks.
2. The question paper consists of two sections:
 - Section A: Objective-type questions.
 - Section B: Programming-based questions with problem statements.
3. Students are required to write, execute, and test the complete program for each problem statement.
4. Clearly display the output of each program. Include intermediate results wherever applicable.
5. Assume appropriate input values, datasets, and parameter values wherever required unless otherwise specified.
6. Students may use any IDE or development platform such as Visual Studio Code (VS Code), Cursor, Google Colab, Jupyter Notebook, PyCharm, Spyder, or any other equivalent platform they are comfortable with.
7. Use appropriate variable names, comments, and proper indentation to improve code readability.
8. Ensure that the program is error-free and executable.
9. Save all source code files properly before submission.

Submission Instructions

After completing all programming questions in **Section B**:

1. Open **Microsoft Word (MS Word)**.
2. Prepare a document containing the following details on the first page:
 - Name
 - Roll Number
 - Branch
 - Section
 - Workshop/Lab Number
3. Copy and paste the **source code** of each program into the document.
4. Insert the corresponding **output screenshots** below each program.
5. Ensure that all screenshots are **clear and readable**.
6. Save the document in **PDF** format.
7. Name the PDF file using the following format:

NAME_ROLLNUMBER_BRANCH_SECTION.pdf

Example:

RahulSharma_2412345678_CSEAIML_A.pdf

8. Upload the PDF file to **Microsoft Forms** before the submission deadline.

Section B**QUESTION 1: Fine-Tuning a Pre-trained CNN Model [CO2: K4]****Problem Statement**

A medical imaging company wants to improve the performance of an image classification model by adapting a pre-trained network to a new dataset. Implement **Fine-Tuning** using a pre-trained **ResNet50** or **VGG16** model on the **CIFAR-10 Dataset**. Initially freeze the base model, then unfreeze the last few convolutional layers for fine-tuning, and evaluate the model performance.

Given Parameters

Parameter	Value
Dataset	CIFAR-10
Image Size	224 × 224 (Resize from 32 × 32)
Number of Classes	10
Base Model	ResNet50 / VGG16
Optimizer	Adam
Learning Rate	0.0001
Loss Function	Categorical Cross Entropy
Epochs	5
Validation Split	20%

Tasks

1. Load the **CIFAR-10** dataset.
2. Display at least **10 sample images** with their corresponding class labels.
3. Resize the images from **32 × 32** to **224 × 224**.
4. Load the pre-trained **ResNet50** or **VGG16** model with **ImageNet** weights.
5. Freeze all convolutional layers and train the newly added classifier.
6. Unfreeze the last few convolutional layers and perform **Fine-Tuning**.
7. Evaluate the model on the testing dataset and report the classification accuracy.
8. Display predictions for the first **10 testing images** and compare them with the actual class labels.

QUESTION 2: Transfer Learning using Pre-Trained Model: [CO2: K4]

Problem Statement

A bank wants to automate the recognition of handwritten digits on cheques to improve processing speed and accuracy. Implement **Transfer Learning** using a pre-trained **ResNet50** or **VGG16** model on the **MNIST Handwritten Digit Dataset**. Replace the final classification layer to classify handwritten digits and evaluate the model performance.

Given Parameters

Parameter	Value
Dataset	MNIST Handwritten Digit Dataset
Image Size	224 × 224 (Resize from 28 × 28)
Image Type	Grayscale (Convert to RGB if required)
Number of Classes	10
Base Model	ResNet50 / VGG16
Optimizer	Adam
Loss Function	Sparse Categorical Crossentropy
Epochs	5
Validation Split	20%

Tasks

1. Load the **MNIST Handwritten Digit Dataset**.
2. Display at least **10 sample handwritten digit images** with their corresponding class labels.
3. Resize the images from **28 × 28** to **224 × 224** and convert the grayscale images to **RGB**.
4. Load the pre-trained **ResNet50** or **VGG16** model with **ImageNet** weights.
5. Freeze all the layers of the pre-trained model and replace the final classification layer with a new classifier for **10 output classes**.
6. Train the transfer learning model using the **Adam** optimizer.
7. Evaluate the model on the testing dataset and report the classification accuracy.
8. Display predictions for the first **10 testing images** and compare the predicted class labels with the actual class labels.

QUESTION 3: Object Detection using YOLOv8 [CO3: K4]**Problem Statement**

A smart traffic monitoring system needs to detect and identify multiple objects such as cars, buses, bicycles, and pedestrians in road images. Implement **YOLOv8** for object detection using the **COCO128 Dataset** and display the detected objects with their bounding boxes and confidence scores.

Parameter	Value
Dataset	COCO128
Model	YOLOv8n
Image Size	640 × 640
Number of Classes	80
Optimizer	SGD (Default)
Epochs	10
Confidence Threshold	0.25

Tasks

1. Load the **COCO128** dataset.
2. Load the pre-trained **YOLOv8n** model.
3. Train the model on the dataset.
4. Perform object detection on at least **5 testing images**.
5. Display the detected objects with **bounding boxes**, **class labels**, and **confidence scores**.
6. Evaluate the model using **Precision**, **Recall**, and **mAP (Mean Average Precision)**.
7. Compare the predicted objects with the ground truth annotations.

QUESTION 4: Object Detection using SSD: [CO3: K6]

Problem Statement

A smart city surveillance system needs to detect and classify multiple objects such as persons, cars, buses, bicycles, traffic lights, and animals from street images in real time. Implement the Single Shot Detector (SSD) model using the COCO Dataset to perform object detection and evaluate the model's performance.

Given Parameters

Parameter	Value
Dataset	COCO Dataset (Common Objects in Context)
Base Model	SSD300 (VGG16 Backbone)
Image Size	300 × 300
Number of Classes	80
Optimizer	Adam
Loss Function	MultiBox Loss
Epochs	10
Confidence Threshold	0.50

Tasks

1. Load the **COCO Dataset**.
2. Display at least **5 sample images** with their corresponding ground truth bounding boxes and class labels.
3. Load the pre-trained **SSD300** model with the **VGG16** backbone.
4. Train or fine-tune the SSD model using the COCO dataset.
5. Perform object detection on at least **5 testing images**.
6. Display the detected objects with **bounding boxes**, **class labels**, and **confidence scores**.
7. Evaluate the model using **Precision**, **Recall**, and **Mean Average Precision (mAP)**.
8. Compare the predicted bounding boxes with the corresponding ground truth annotations.

Wishing you all the best